BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2
NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN, ĐIỆN TỬ

BÁO CÁO ĐỒ ÁN HỌC PHẦN

MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI:

SẮP XẾP LỰA CHỌN VÀ TIẾT KIỆM ĐỔI CHỖ

Mã đề: DT011

Giảng viên hướng dẫn : Hồ Nhựt Minh
Sinh viên thực hiện : TRẦN HỮU DUY
MSSV : N24DCDT022
Lớp : D24CQDT01-N

TP. HCM, tháng 10 năm 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Nội dung
Họ tên MSSV lớp	Trần Hữu Duy — N24DCDT022 — D24CQDT01-N
Mã đề và tên đề	DT011 — Sắp xếp lựa chọn và tiết kiệm đổi chỗ
Căn cứ giáo trình	Chương 2, Mục 2.2.2; bảng 2.2; mục 2.8 bài 3, 11, 19; trang in 35–37, 60–61; trang PDF 47–49, 72–73
Ngày giao ngày nộp	Ngày giao 17/10/2026 — Ngày nộp 23/10/2026

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý

tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

LỜI CAM ĐOAN

Tôi xin cam đoan toàn bộ nội dung báo cáo này do chính tôi thực hiện: tự đọc giáo trình, tự cài đặt thuật toán sắp xếp lựa chọn, tự viết bộ kiểm thử, tự chạy và chụp bằng chứng. Các đoạn mã, bảng truy vết và kết quả là sản phẩm của riêng tôi, không sao chép của người khác. Mã mẫu của giảng viên chỉ dùng để tham khảo cách tổ chức và đã nêu ở tài liệu tham khảo.

TP. Hồ Chí Minh, ngày 23 tháng 10 năm 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn giảng viên Hồ Nhựt Minh đã hướng dẫn, giao đề tài, cung cấp khung báo cáo và tài liệu tham khảo để em hoàn thành đồ án học phần Cấu trúc dữ liệu và Giải thuật.

TÓM TẮT

Đồ án trình bày thuật toán sắp xếp lựa chọn (selection sort) trên mảng số nguyên: ở mỗi lượt tìm phần tử nhỏ nhất của đoạn chưa sắp rồi đổi về đầu đoạn, đồng thời đếm số phép so sánh và số phép hoán đổi thực sự. Kết quả cho thấy thuật toán luôn thực hiện n(n-1)/2 phép so sánh nhưng số phép đổi chỗ không quá n-1, ít hơn hẳn thuật toán đổi chỗ cùng bậc hai O(n^2). Sản phẩm gồm mã nguồn Julia, bộ kiểm thử 14 ca độc lập, 11 ảnh minh chứng và báo cáo theo mẫu.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG ii
CÁCH ĐỐI CHIẾU RUBRIC iii
LỜI CAM ĐOAN v
LỜI CẢM ƠN vi
TÓM TẮT vii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3
2.1. Khái niệm và mô hình dữ liệu 3
2.2. Cấu trúc dữ liệu sử dụng 3
2.3. Nguyên lý thuật toán 3
2.4. Cơ sở đánh giá thời gian và bộ nhớ 4
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 5
3.1. Phân tích yêu cầu chương trình 5
3.2. Thiết kế các hàm và dữ liệu 5
3.3. Thuật toán và mã nguồn chính 6
3.4. Lần vết ví dụ nhỏ 7
3.5. Giải thích tính đúng và điều kiện dừng 8
3.6. Phân tích độ phức tạp 8
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 9
4.1. Môi trường và kế hoạch kiểm thử 9
4.2. Bảng kết quả 14 ca 9
4.3. Hai ví dụ trên phiếu 13
4.4. Hai ca sinh viên tự thiết kế 14
4.5. Ba ca biên phù hợp với đề tài 15
4.6. Kiểm tra input không hợp lệ — LOI01 16
4.7. Kết quả toàn bộ kiểm thử 16
4.8. Khảo sát bốn quy mô 16
4.9. GitHub và chạy lại từ bản clone tại commit C 18
4.10. Mô phỏng trực quan (tuỳ chọn) 20
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 21
5.1. Mức hoàn thành so với phiếu 21
5.2. Giới hạn và bài học 21
5.3. Hướng phát triển phù hợp 21
TÀI LIỆU THAM KHẢO 22
PHỤ LỤC PL-1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy PL-1
PHỤ LỤC B. Danh mục bằng chứng PL-2
PHỤ LỤC C. Tự kiểm trước khi nộp PL-3
PHỤ LỤC D. THÔNG TIN NỘP BÀI PL-4

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào.....2
Bảng 1.2 Đặc tả kết quả đầu ra.....2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm.....2
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì.....3
Bảng 3.1 Tổ chức các hàm và tệp.....5
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm.....6
Bảng 3.3 Truy vết ca T01 (VD01).....7
Bảng 3.4 Chi phí của từng pha trong bản cài.....8
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu.....9
Bảng 4.2 Thiết kế 14 ca kiểm độc lập.....10
Bảng 4.3 Actual kết luận và vị trí minh chứng.....12
Bảng 4.4 Khảo sát bốn quy mô Q1–Q4.....17
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại.....19
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu.....22
Bảng PL.1 Bốn tệp cần nộp.....PL-4

DANH SÁCH HÌNH

Hình 4.1 Ví dụ 1 (VD01)....13
Hình 4.2 Ví dụ 2 (VD02)....13
Hình 4.3 Ca tự thiết kế 1 (TK01)....14
Hình 4.4 Ca tự thiết kế 2 (TK02)....14
Hình 4.5 Biên rỗng (BIEN01)....15
Hình 4.6 Một phần tử (BIEN02)....15
Hình 4.7 Biên toàn trùng (BIEN03)....16
Hình 4.8 Input không hợp lệ (LOI01)....16
Hình 4.9 Tổng kết 14 ca (CHECK)....17
Hình 4.10 Clone C (e0db818)....20
Hình 4.11 GitHub repo (e0db818)....20

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Đề tài "Sắp xếp lựa chọn và tiết kiệm đổi chỗ" yêu cầu sắp xếp một mảng số nguyên tăng dần bằng thuật toán sắp xếp lựa chọn: ở mỗi lượt tìm phần tử nhỏ nhất của đoạn chưa sắp rồi đổi về đầu đoạn. Trọng tâm là theo dõi chỉ số min, đếm số phép so sánh và số phép hoán đổi thực sự, từ đó so với thuật toán đổi chỗ (mục 2.2.1) để giải thích vì sao hai thuật toán cùng bậc hai nhưng khác lượng thay đổi mảng. Căn cứ: Chương 2, Mục 2.2.2, bảng 2.2, trang in 35–37.

1.2. Mục tiêu và sản phẩm

Các mục tiêu và sản phẩm:

- Cài đặt sap_xep_lua_chon(a) trả về mảng tăng dần, số so sánh, số đổi chỗ thực và vị trí min từng lượt.
- Cài đặt hàm con tim_min_doan(a, l, r) tìm vị trí nhỏ nhất trong một đoạn.
- Cài đặt lan_vet_lua_chon(a) in từng lượt theo bảng 2.2.
- Đối chứng sap_xep_doi_cho(a) để so số đổi chỗ.
- Bộ kiểm thử ≥12 ca độc lập, 11 ảnh minh chứng, báo cáo và kho mã chạy lại được.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Dữ liệu vào là mảng a kiểu Vector{Int} với n phần tử, $0 \le n \le 2000$ và $|a[i]| \le 10^6$. Phần tử trùng nhau hợp lệ. Dữ liệu nhập từ bàn phím hoặc khai báo trực tiếp, không đọc tệp CSV/Excel. Chỉ số mảng dùng 1-based theo quy ước của Julia.

1.3.2. Kết quả và trường hợp đặc biệt

Kết quả gồm: mảng đã sắp tăng dần; số phép so sánh; số phép hoán đổi thực sự (bỏ qua phép đổi một ô với chính nó khi min_index == i); danh sách vị trí min từng lượt. Không có trường hợp vô nghiệm: mọi mảng hợp lệ đều sắp được. Input sai bị từ chối bằng ArgumentError.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
a (mảng đầu vào)	Vector{Int}, n phần tử	$0 \le n \le 2000$; $\|a[i]\| \le 10^6$; trùng hợp lệ	Khai báo trực tiếp / bàn phím
n (số phần tử)	Số nguyên không âm	$0 \le n \le 2000$	Suy từ độ dài mảng a
l, r (chỉ số đoạn)	Số nguyên 1-based	$1 \le l \le r \le n$	Tham số hàm tim_min_doan

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
Mảng đã sắp	Vector{Int}	Tăng dần, đúng đa tập của a	Không áp dụng hòa/vô nghiệm
Số so sánh, đổi chỗ, vị trí min	Số nguyên / danh sách	so sánh = n(n-1)/2; đổi chỗ ≤ n-1	Input sai: ArgumentError

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số; thuật toán không phụ thuộc CSV/Excel hay tệp ngoài. Trường hợp hòa/vô nghiệm không áp dụng: mọi mảng hợp lệ có đúng một kết quả; chỉ input sai mới bị từ chối.

1.4. Cách thực hiện và cấu trúc báo cáo

Quy trình: đọc giáo trình mục 2.2.2 → giải ví dụ bằng tay → cài đặt → chạy kiểm thử → chụp bằng chứng. Báo cáo gồm năm chương như đã liệt kê ở mục lục.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Cài đặt sắp xếp lựa chọn	Hàm sap_xep_lua_chon chạy đúng mọi ca	Chương 3, mục 3.3
Theo dõi min, so sánh, đổi chỗ	Đếm đúng n(n-1)/2 so sánh, ≤ n-1 đổi chỗ	Chương 3, mục 3.3–3.4
Đối chứng thuật toán đổi chỗ	So số đổi chỗ hai thuật toán	Chương 4, mục 4.4.2

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Đối tượng chính là mảng số nguyên a[1..n], n là số phần tử. Đoạn chưa sắp là dãy con a[i..n] với i là vị trí bắt đầu lượt hiện hành. Phần tử nhỏ nhất của đoạn xác định bởi chỉ số min thỏa a[min] ≤ a[j] với mọi j trong đoạn. Giáo trình trình bày ở mục 2.2.2, bảng 2.2 (trang in 35–37).

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Cấu trúc dùng là mảng (Vector{Int}). Thao tác chính: đọc a[j], so sánh a[j] < a[min] và hoán đổi a[i], a[min]. Mảng phù hợp vì đề chỉ cần sắp xếp tại chỗ, truy cập ngẫu nhiên O(1), không cần cấu trúc phụ. Bản sao b = copy(a) giữ nguyên dữ liệu gốc của người gọi.

2.2.2. Quy ước và điều kiện hợp lệ

Quy ước: chỉ số 1-based; mảng rỗng Int[] có n = 0; phần tử trùng hợp lệ, khi nhiều phần tử cùng nhỏ nhất giữ chỉ số đầu tiên (chỉ cập nhật khi a[j] < a[min]); bỏ hoán đổi khi min_index == i. Các khái niệm cây, đồ thị không dùng trong đề này.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
a[1..n]	Mảng cần sắp	|a[i]| ≤ 10^6; giữ nguyên đa tập
i (đầu đoạn chưa sắp)	Biến vòng ngoài	Sau lượt i, a[1..i] cố định đúng
min_idx	Vị trí nhỏ nhất đoạn a[i..n]	a[min_idx] ≤ a[j] với mọi j trong [i..n]

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Ở mỗi lượt i (1..n-1) thuật toán quét đoạn a[i..n] tìm vị trí min rồi đổi a[i] với a[min] nếu min khác i. Sau lượt i, phần tử nhỏ nhất còn lại nằm đúng vị trí i. Đây là điểm khác với sắp xếp đổi chỗ: đổi chỗ hoán đổi ngay mỗi khi gặp phần tử nhỏ hơn nên số lần đổi nhiều hơn hẳn.

2.3.2. Điều kiện bắt đầu và kết thúc

Khởi tạo b = copy(a), so_sanh = 0, so_doi = 0, min_moi_luot rỗng. Vòng ngoài i = 1..n-1, vòng trong j = i+1..n. Dừng sau lượt i = n-1 (đoạn còn một phần tử). Trả về mảng đã sắp cùng các bộ đếm. Lưu ý tính ổn định: sắp xếp lựa chọn KHÔNG ổn định. Với [2a,2b,1], lượt 1 tìm min là 1 tại vị trí 3 rồi đổi a[1] với a[3], thu được [1,2b,2a] — hai phần tử bằng nhau bị đảo thứ tự tương đối. Muốn ổn định phải dùng sắp xếp chèn/trộn hoặc thêm điều kiện chọn.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Phép toán chính là so sánh a[j] < a[min]. Số so sánh lượt i là n-i, tổng là n(n-1)/2 nên thời gian O(n^2) cho mọi dữ liệu. Số hoán đổi tối đa n-1. Bộ nhớ phụ O(1) cho biến đếm; bản sao b tốn O(n) (không bắt buộc nếu sắp tại chỗ). Dự đoán n = 5, 20, 100, 500 tăng bậc hai.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Hợp đồng các hàm (tên biến khớp mã nguồn):

- kiem_tra_dau_vao(a) → true nếu hợp lệ, ngược lại ném ArgumentError.
- tim_min_doan(a, l, r) → chỉ số min trong a[l..r].
- sap_xep_lua_chon(a) → (mang, so_sanh, so_doi, min_moi_luot).
- lan_vet_lua_chon(a) → in từng lượt, trả cùng kết quả.
- sap_xep_doi_cho(a) → (mang, so_sanh, so_doi) để đối chứng.

3.1.2. Kiểm tra đầu vào

Cần xác thực: a là Vector{Int}; n ≤ 2000; |a[i]| ≤ 10^6. Khi vi phạm ném ArgumentError kèm lý do. Phân biệt rõ: mảng rỗng là input hợp lệ (n = 0, kết quả rỗng), không phải input sai.

3.2. Thiết kế các hàm và dữ liệu

Các hàm đều tự cài bằng vòng lặp, không dùng sort có sẵn trong lõi; sort chỉ dùng làm oracle độc lập trong kiểm thử. Phần tự cài gồm vòng quét tìm min, hoán đổi có điều kiện và bộ đếm.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
sap_xep_lua_chon(a)	Vector{Int} a	KetQua (mang, so_sanh, so_doi, min_moi_luot)	Lõi thuật toán (tự cài)
tim_min_doan(a, l, r)	a, l, r	Int (vị trí min)	Hàm con tìm min (tự cài)
lan_vet_lua_chon(a)	Vector{Int} a	In từng lượt; trả KetQua	Lần vết theo bảng 2.2
sap_xep_doi_cho(a)	Vector{Int} a	KetQua (mang, so_sanh, so_doi)	Đối chứng (tự cài)

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
tim_min_doan(a, l, r)	$1 \le l \le r \le n$	Trả min_idx: a[min_idx] ≤ a[j]	l > r ném ArgumentError
sap_xep_lua_chon(a)	a hợp lệ	mang tăng; so_sanh = n(n-1)/2; so_doi ≤ n-1	Không đổi a gốc (dùng bản sao)
kiem_tra_dau_vao(a)	a bất kỳ	true khi hợp lệ	Ném ArgumentError khi vi phạm
sap_xep_doi_cho(a)	a hợp lệ	mang tăng; cùng so_sanh, so_doi nhiều hơn	Đối chứng số đổi chỗ

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

```
Giả mã sap_xep_lua_chon:
  b = copy(a); so_sanh = 0; so_doi = 0; min_moi_luot = []
  for i = 1 .. n-1
      min_idx = i
      for j = i+1 .. n
          so_sanh += 1
          if b[j] < b[min_idx]: min_idx = j
      push!(min_moi_luot, min_idx)
      if min_idx != i: hoán đổi b[i], b[min_idx]; so_doi += 1
  return (b, so_sanh, so_doi, min_moi_luot)
```

3.3.2. Đoạn mã và giải thích

Đoạn mã quyết định nằm trong src/DT011.jl: hai vòng lặp lồng nhau, vòng ngoài chọn i, vòng trong quét tìm min. Phép hoán đổi chỉ xảy ra sau khi quét xong và chỉ khi min_idx != i, đảm bảo không tính phép đổi một ô với chính nó. Permalink tại commit C ghi ở Bảng 4.5.

```
i = 0
while i < n - 1
    i += 1
    chon = i
    j = i
    while j < n
        j += 1
        so_sanh += 1
        if giam ? (b[j] > b[chon]) : (b[j] < b[chon])
            chon = j
        end
    end
    if chon != i
        b[i], b[chon] = b[chon], b[i]
        so_doi += 1
    end
end
```

3.3.3. Xử lý tình huống đặc biệt

Tình huống đặc biệt: n = 0, 1 thì vòng ngoài không chạy, trả mảng gốc với 0 so sánh, 0 đổi chỗ; mảng đã tăng cho so_doi = 0; mảng toàn phần tử bằng nhau giữ nguyên; input sai bị từ chối ở kiem_tra_dau_vao trước khi vào lõi (kiểm ở Chương 4: BIEN01–BIEN03, LOI01).

3.4. Lần vết ví dụ nhỏ

Ca truy vết T01 dùng input của Ví dụ 1: a = [8,3,5,1,9,2], expected [1,2,3,5,8,9].

Bảng 3.3 Truy vết ca T01 (VD01)

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Khởi tạo	b = copy(a)	[8,3,5,1,9,2]	n=6, so_sanh=0, so_doi=0	
Lượt 1	Quét [1..6], min tại 4	[8,3,5,1,9,2]	[1,3,5,8,9,2]	Đổi a[1]<->a[4] (8,1)
Lượt 2	Quét [2..6], min tại 6	[1,3,5,8,9,2]	[1,2,5,8,9,3]	Đổi a[2]<->a[6] (3,2)
Lượt 3	Quét [3..6], min tại 6	[1,2,5,8,9,3]	[1,2,3,8,9,5]	Đổi a[3]<->a[6] (5,3)
Lượt 4	Quét [4..6], min tại 6	[1,2,3,8,9,5]	[1,2,3,5,9,8]	Đổi a[4]<->a[6] (8,5)
Lượt 5	Quét [5..6], min tại 6	[1,2,3,5,9,8]	[1,2,3,5,8,9]	Đổi a[5]<->a[6] (9,8)
Cuối	Trả kết quả		[1,2,3,5,8,9]	15 so sánh, 5 đổi chỗ

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Bất biến vòng lặp: trước lượt i, đoạn a[1..i-1] đã cố định đúng (i-1 phần tử nhỏ nhất theo thứ tự tăng). Sau khi quét đoạn a[i..n] và đổi a[i] với phần tử nhỏ nhất, bất biến được duy trì cho lượt i+1. Cơ sở i = 1 đúng (đoạn rỗng); bước chuyển đúng vì phần tử nhỏ nhất còn lại được đưa về đúng vị trí.

3.5.2. Kết thúc và output

Vòng ngoài tăng i từ 1 đến n-1 nên dừng sau đúng n-1 lượt. Khi kết thúc, a[1..n-1] đã cố định đúng, phần tử còn lại a[n] tự là lớn nhất, do đó toàn mảng tăng dần. Kiểm thử Chương 4 là bằng chứng bổ sung, không thay lập luận.

3.6. Phân tích độ phức tạp

Mỗi lượt i thực hiện n-i phép so sánh, tổng n(n-1)/2, nên thời gian O(n^2) không phụ thuộc dữ liệu. Số hoán đổi tối đa n-1. Bộ nhớ phụ O(1) cho biến đếm; bản sao b tốn O(n).

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm tra và chuẩn bị	n	Kiểm tra n, |a[i]|	O(n)	O(1)
Thuật toán chính	n	n(n-1)/2 phép so sánh	O(n^2)	O(1)
Bản sao / đối chứng	n	copy(a), sap_xep_doi_cho	O(n^2)	O(n)
In và định dạng	n	In mảng, bộ đếm	O(n)	O(1)
Toàn chương trình	n	—	O(n^2)	O(n)

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Môi trường: Windows 11; VSCode 1.142.0 (Antigravity) dùng terminal PowerShell; Julia 1.13.1 (chỉ thư viện chuẩn). Mã chính tại src/DT011.jl, CLI tại bin/dt011.jl, bộ kiểm thử tại spec/runtests.jl, khảo sát tại bench/scales.jl. Commit chạy và chụp ảnh là C e0db818; commit nộp S ghi trong ThongTinNop.txt.

4.1.2. Phương pháp xác định kỳ vọng

Kỳ vọng được tính độc lập bằng tay (giải ví dụ từng bước) và đối chiếu bằng hàm sort có sẵn của Julia trong kiểm thử (oracle độc lập, không dùng trong lõi thuật toán). Với mảng, kết quả là duy nhất nên so sánh trực tiếp không cần sai số.

4.2. Bảng kết quả 14 ca

Bảng dưới liệt kê 14 ca kiểm thử độc lập, gồm hai ví dụ và toàn bộ năm ca trên phiếu, bổ sung các ca tự thiết kế, biên và input sai. Mỗi ca có mã, input đủ tái hiện, expected tính độc lập và actual đối chiếu.

Miền chính là mảng số nguyên; các ca biên và ca hàm con (tim_min_doan, validator) được dùng để kiểm nhánh đặc biệt của thuật toán.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	[1,2,3,5,8,9] đã tăng (dãy kết quả VD01)	BIEN04 / demo_all.log	evidence/logs/ demo_all.log
Ca phiếu 2	[2,1,2] cho [1,2,2]	VD02 / A02	02_VD02.png
Ca phiếu 3	[1,2,3] không đổi chỗ	BIEN04 / demo_all.log	evidence/logs/ demo_all.log
Ca phiếu 4	[4,4,4] giữ đủ 3 phần tử	BIEN03 / demo_all.log	evidence/logs/ demo_all.log
Ca phiếu 5	Mảng rỗng Int[] / một phần tử [7] không lỗi	BIEN01 + BIEN02 / A05 + A06	05_BIEN01.png; 06_BIEN02.png

Ca phiếu 1 ([1,2,3,5,8,9]) là dãy đã tăng — chính là kết quả của VD01; tính chất "đã tăng thì không đổi chỗ" được kiểm trực tiếp bằng BIEN04 trên [1,2,3] (cùng mục tiêu nên đếm một lần), còn giá trị đúng của dãy 6 phần tử đã được xác nhận ở VD01. Ca phiếu 5 được phủ bởi hai ca: BIEN01 (Int[]) và BIEN02 ([7]).

Bảng 4.2 Thiết kế 14 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ 1 phiếu / sap_xep_lua_chon	[8,3,5,1,9,2]	ra [1,2,3,5,8,9]; so_sanh=15; so_doi=5
VD02	Ví dụ 2 phiếu / sap_xep_lua_chon	[2,1,2]	ra [1,2,2]; so_sanh=3; so_doi=1
TK01	Tự thiết kế 1 (trùng 1) / sap_xep_lua_chon	[5,1,4,1,3]	ra [1,1,3,4,5]; so_doi=4
TK02	Tự thiết kế 2 (giảm dần) / sap_xep_lua_chon	[9,7,5,3,1]	ra [1,3,5,7,9]; so_doi=2
BIEN01	Biên rỗng / sap_xep_lua_chon	Int[]	ra Int[]; so_sanh=0; so_doi=0
BIEN02	Biên một phần tử / sap_xep_lua_chon	[7]	ra [7]; so_sanh=0; so_doi=0
BIEN03	Biên toàn trùng / sap_xep_lua_chon	[4,4,4]	ra [4,4,4]; so_sanh=3; so_doi=0
BIEN04	Biên đã tăng / sap_xep_lua_chon	[1,2,3]	ra [1,2,3]; so_sanh=3; so_doi=0
KT01	Số âm xen kẽ / sap_xep_lua_chon	[3,-1,3,-1,0]	ra [-1,-1,0,3,3]; so_doi=3
KT02	Hai đầu biên 10^6 / sap_xep_lua_chon	[1000000,-1000000,0]	ra [-1000000,0,1000000]; so_doi=2
KT03	n=2 nghịch thế / sap_xep_lua_chon	[2,1]	ra [1,2]; so_sanh=1; so_doi=1
KT04	100 phần tử bằng nhau / sap_xep_lua_chon	fill(1,100)	ra 1×100; so_sanh=4950; so_doi=0
KT05	50 phần tử sinh theo công thức / sap_xep_lua_chon	mod(37i,201)-100	đối chiếu sort; so_sanh=1225

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
LOI01	Input sai / kiem_tra_dau_vao	n=2001; |a[i]|=10^6+1; Float64	ném ArgumentError cả 3 kiểu

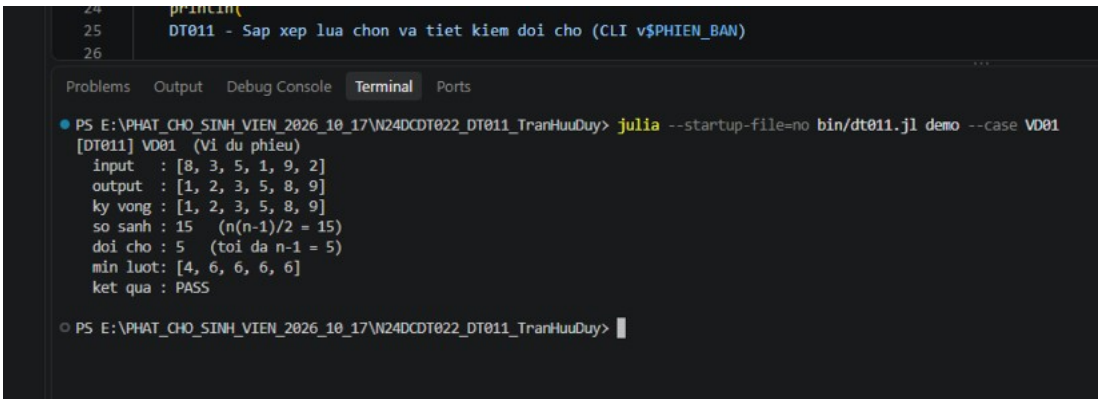
Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh và log minh chứng
VD01	[1,2,3,5,8,9]	PASS	A01 / 01_VD01.png
VD02	[1,2,2]	PASS	A02 / 02_VD02.png
TK01	[1,1,3,4,5]	PASS	A03 / 03_TK01.png
TK02	[1,3,5,7,9]	PASS	A04 / 04_TK02.png
BIEN01	[]	PASS	A05 / 05_BIEN01.png
BIEN02	[7]	PASS	A06 / 06_BIEN02.png
BIEN03	[4,4,4]	PASS	demo_all.log
BIEN04	[1,2,3]	PASS	demo_all.log
KT01	[-1,-1,0,3,3]	PASS	A09 / 09_check.png
KT02	[-1000000,0,1000000]	PASS	A09 / 09_check.png
KT03	[1,2]	PASS	A09 / 09_check.png
KT04	1×100	PASS	A09 / 09_check.png
KT05	50 phần tử đã sắp	PASS	A09 / 09_check.png
LOI01	ArgumentError	PASS	A08 / 08_LOI01.png

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Input: [8,3,5,1,9,2]. Expected: [1,2,3,5,8,9], xác định bằng giải tay và sort độc lập. Lệnh chạy: julia --startup-file=no bin/dt011.jl demo --case VD01. Actual: [1,2,3,5,8,9], 15 so sánh, 5 đổi chỗ, min từng lượt [4,6,6,6,6].



Hình 4.1 — Ví dụ 1 (VD01): output thật từ bin/dt011.jl demo --case VD01.

4.3.2. Ví dụ 2 — VD02

Input: [2,1,2]. Expected: [1,2,2], xác định bằng giải tay (giữ đủ hai phần tử 2). Lệnh chạy: julia --startup-file=no bin/dt011.jl demo --case VD02. Actual: [1,2,2], 3 so sánh, 1 đổi chỗ, min [2,2].

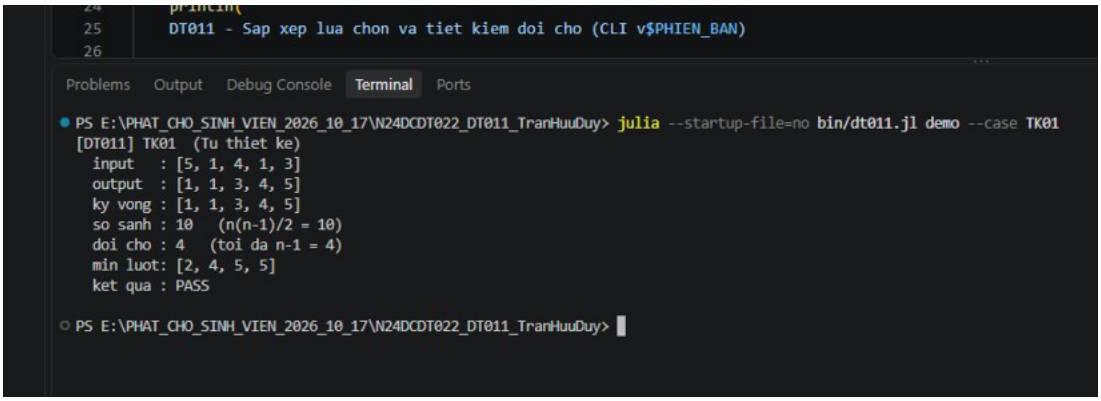


Hình 4.2 — Ví dụ 2 (VD02): output thật từ bin/dt011.jl demo --case VD02.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

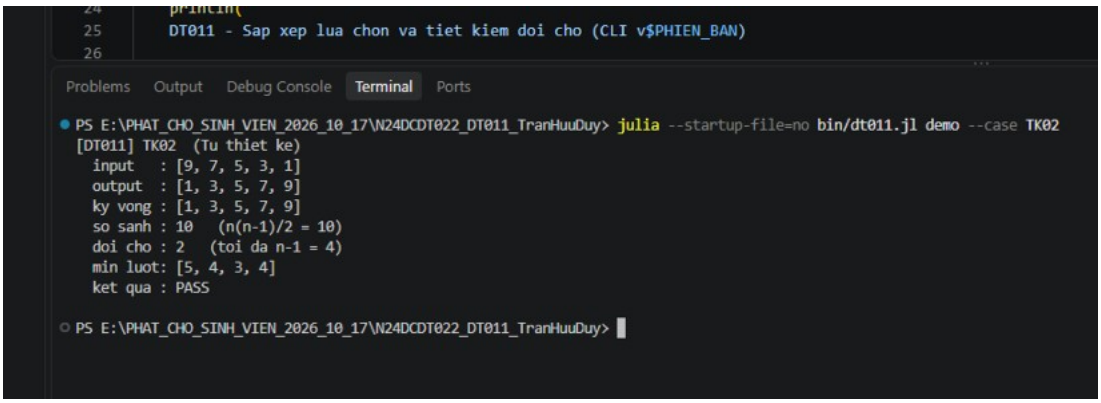
Mục đích: kiểm phần tử trùng. Input [5,1,4,1,3], expected [1,1,3,4,5]. Actual khớp: 10 so sánh, 4 đổi chỗ. Hai số 1 trùng giá trị nên không phân biệt được thứ tự; sắp xếp lựa chọn nói chung KHÔNG ổn định — ví dụ [2a,2b,1] cho [1,2b,2a] vì lượt 1 đổi a[1] với a[3]. Tính không ổn định được phân tích ở mục 2.3.2.



Hình 4.3 — Ca tự thiết kế 1 (TK01).

4.4.2. Ca TK02

Mục đích: mảng giảm dần — trường hợp tiết kiệm đổi chỗ rõ nhất. Input [9,7,5,3,1], expected [1,3,5,7,9]. Actual khớp: 10 so sánh nhưng chỉ 2 đổi chỗ. Chạy đối chứng sap_xep_doi_cho trên cùng input cho 10 đổi chỗ (đúng bằng số nghịch thế; log tại evidence/logs/so_sanh.log, chạy tại C); hai mảng khác cũng lệch rõ: [8,3,5,1,9,2] là 5 so với 9, [5,1,4,1,3] là 4 so với 6. Hàm đối chứng được kiểm trong testset “Doi chung sap xep doi cho” và “Ham con tim_min_doan” với các assert lc.mang == dc.mang == sort(a), lc.so_sanh == dc.so_sanh và lc.so_doi ≤ dc.so_doi.

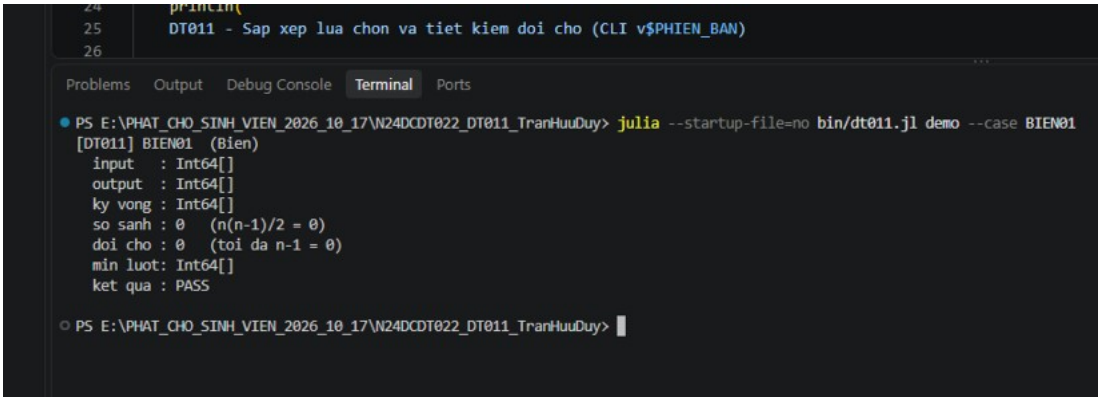


Hình 4.4 — Ca tự thiết kế 2 (TK02).

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01 — mảng rỗng

Biên rỗng: Int[] (n = 0). Expected Int[], so_sanh = 0, so_doi = 0. Actual khớp; đây là ca phiếu 5 (mảng rỗng). Hình 4.5 là ảnh chạy thật ca này.



Hình 4.5 — Biên rỗng (BIEN01): mảng Int[] không lỗi.

4.5.2. Ca BIEN02 — một phần tử

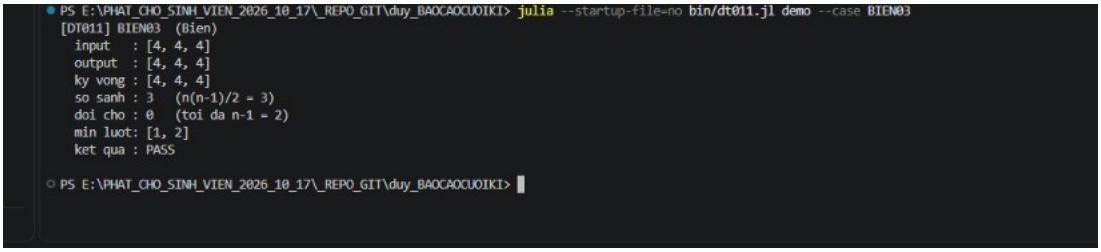
Biên một phần tử: [7] (n = 1). Expected [7], so_sanh = 0, so_doi = 0. Actual khớp; đây là ca phiếu 5 (một phần tử). Hình 4.6 là ảnh chạy thật ca này.



Hình 4.6 — Một phần tử (BIEN02).

4.5.3. Ca BIEN03 — toàn trùng

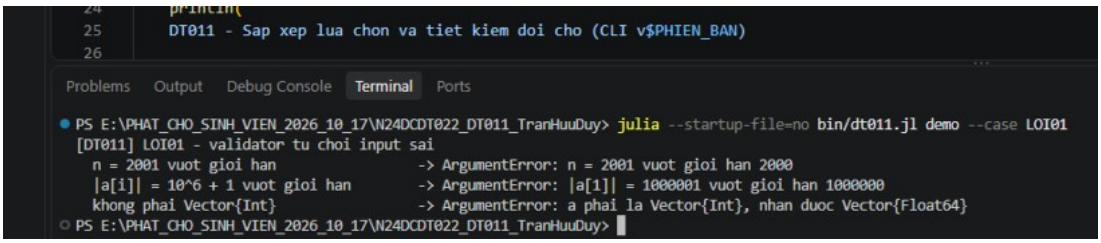
Biên toàn trùng: [4,4,4] (ca phiếu 4). Expected [4,4,4], so_sanh = 3, so_doi = 0 vì min luôn đúng chỗ. Ca BIEN04 ([1,2,3] đã tăng) cũng cho so_doi = 0; cả hai nằm trong evidence/logs/demo_all.log. Hình 4.7 là ảnh chạy ca BIEN03; bảng lần vết VD01 trình bày ở mục 3.4.



Hình 4.7 — Biên toàn trùng (BIEN03): [4,4,4] giữ nguyên, 3 so sánh, 0 đổi chỗ.

4.6. Kiểm tra input không hợp lệ — LOI01

Điều kiện phiếu bị vi phạm: n = 2001 > 2000; |a[i]| = 10^6+1 > 10^6; hoặc không phải Vector{Int}. Expected: hàm kiem_tra_dau_vao ném ArgumentError kèm lý do. Actual: ném đúng ArgumentError, không vào lõi thuật toán.



Hình 4.8 — Input không hợp lệ (LOI01).

4.7. Kết quả toàn bộ kiểm thử

Chạy lại toàn bộ 14 ca: 14/14 PASS. Bộ kiểm thử spec/runtests.jl có 137 phép kiểm, tất cả đạt, không còn lỗi.



Hình 4.9 — Tổng kết 14 ca: bin/dt011.jl check in 137 phép kiểm đạt.

4.8. Khảo sát bốn quy mô

4.8.1. Dữ liệu và điều kiện khảo sát

Khảo sát bốn quy mô n = 5, 20, 100, 500 (theo phiếu) trên mảng ngẫu nhiên |a[i]| ≤ 10^6, seed 20261020 để tái hiện. Phép đếm là số so sánh (tất định = n(n-1)/2) và số đổi chỗ thực của cả lựa chọn lẫn đổi chỗ.

4.8.2. Kết quả và giới hạn nhận xét

Số so sánh đếm thực khớp đúng công thức n(n-1)/2 ở cả bốn quy mô (10; 190; 4.950; 124.750), xác nhận O(n^2). Số đổi chỗ của lựa chọn là 4; 15; 93; 492, còn của sắp xếp đổi chỗ là 6; 111; 2.383; 64.815 — mức chênh lệch tăng rất nhanh theo n, đúng tinh thần "tiết kiệm đổi chỗ". Thời gian đo chỉ mang tính tham khảo vì quy mô còn nhỏ.

Bảng 4.4 Khảo sát bốn quy mô Q1–Q4

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	n = 5, mảng ngẫu nhiên (seed 20261020)	quy mô n	so_sanh = 10; so_doi_lc = 4; so_doi_dc = 6; ~0,0004 ms	O(n) do bản sao
Q2	n = 20, mảng ngẫu nhiên	quy mô n	so_sanh = 190; so_doi_lc = 15; so_doi_dc = 111; ~0,0007 ms	O(n) do bản sao
Q3	n = 100, mảng ngẫu nhiên	quy mô n	so_sanh = 4.950; so_doi_lc = 93; so_doi_dc = 2.383; ~0,0075 ms	O(n) do bản sao
Q4	n = 500, mảng ngẫu nhiên	quy mô n	so_sanh = 124.750; so_doi_lc = 492; so_doi_dc = 64.815; ~0,1573 ms	O(n) do bản sao

Bảng 4.4 dùng đúng bốn quy mô 5, 20, 100, 500 và đo cả hai thuật toán trên cùng mảng. Thời gian đo không dùng để chứng minh bậc phức tạp; phép đếm so sánh (tất định) và cặp số đổi chỗ lựa chọn/đổi chỗ là bằng chứng chính.

Dữ liệu tạo ngẫu nhiên bằng MersenneTwister(seed + n) với seed = 20261020 trong bench/scales.jl (mỗi quy mô n dùng một bộ sinh riêng) để tái hiện được. Số liệu thời gian chỉ minh hoạ, không kết luận vượt bằng chứng.

4.9. GitHub và chạy lại từ bản clone tại commit C

4.9.1. Phiên bản có thể đối chiếu

URL repo, SHA chạy C và permalink hàm chính/test tại C được ghi ở Bảng 4.5; SHA nộp S ghi trong ThongTinNop.txt nộp rời. Ảnh A01–A09 được chụp tại bản làm việc commit 5b08e00, không phải tại C: 5b08e00 là commit sau C, chỉ thêm ảnh minh chứng và log, còn mã nguồn, bộ kiểm thử và bench giống hệt C. Đối chứng bằng lệnh dưới đây, kết quả rỗng:

```
$ git diff --stat e0db818 5b08e00 -- bin src spec bench
(không có kết quả – mã, kiểm thử và bench giống hệt nhau)
```

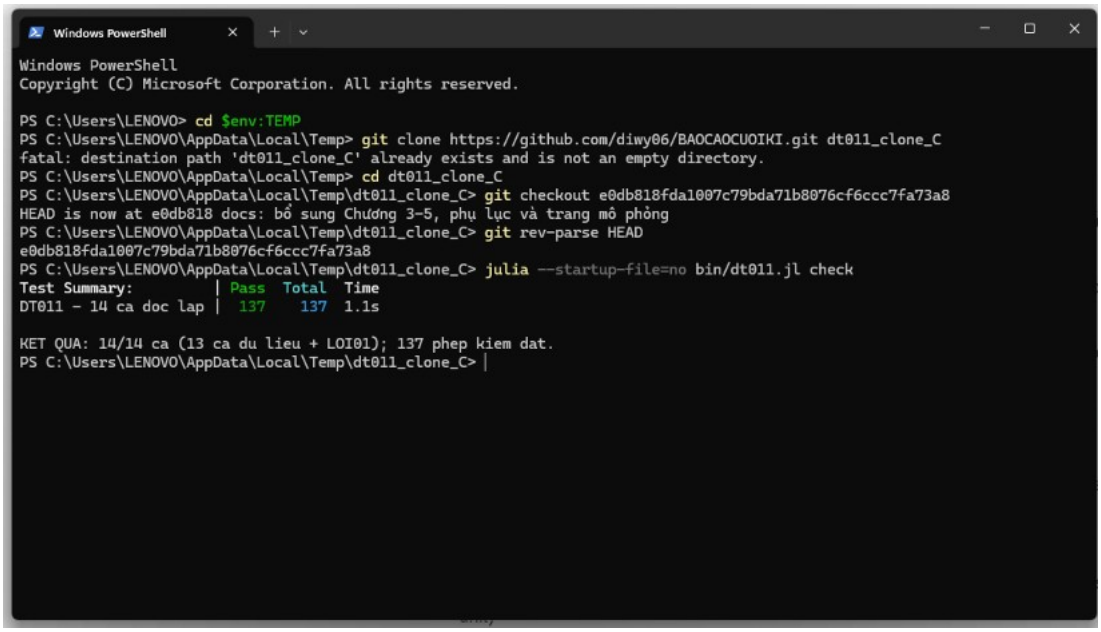
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị
URL repo	https://github.com/diwy06/BAOCAOCUOIKI
Full SHA chạy ảnh C	e0db818fda1007c79bda71b8076cf6ccc7fa73a8
Commit chụp ảnh/log	A01–A09 chụp từ bản làm việc tại commit 5b08e00 (mã giống hệt C — diff rỗng); A10 là bản clone tại C e0db818; A11 là trang commit C trên GitHub.
SHA S ghi TXT nộp rời	Commit S sau khi thêm báo cáo/ảnh — ghi trong ThongTinNop.txt
Permalink hàm chính	https://github.com/diwy06/BAOCAOCUOIKI/blob/e0db818fda1007c79bda71b8076cf6ccc7fa73a8/src/DT011.jl
Mã/kiểm thử giữa C và S	Không đổi — S chỉ thêm báo cáo/ảnh; git diff C S -- bin src spec bench rỗng
Lệnh chạy lại	julia --startup-file=no bin/dt011.jl demo --all ; julia --startup-file=no bin/dt011.jl check

4.9.2. Clone từ GitHub và tái hiện kết quả

Ảnh A10 chụp khi clone kho về thư mục mới bằng git clone, chuyển tới commit C bằng git checkout e0db818, kiểm HEAD rồi chạy lại bộ kiểm thử. Ảnh A11 chụp trang commit C trên GitHub.

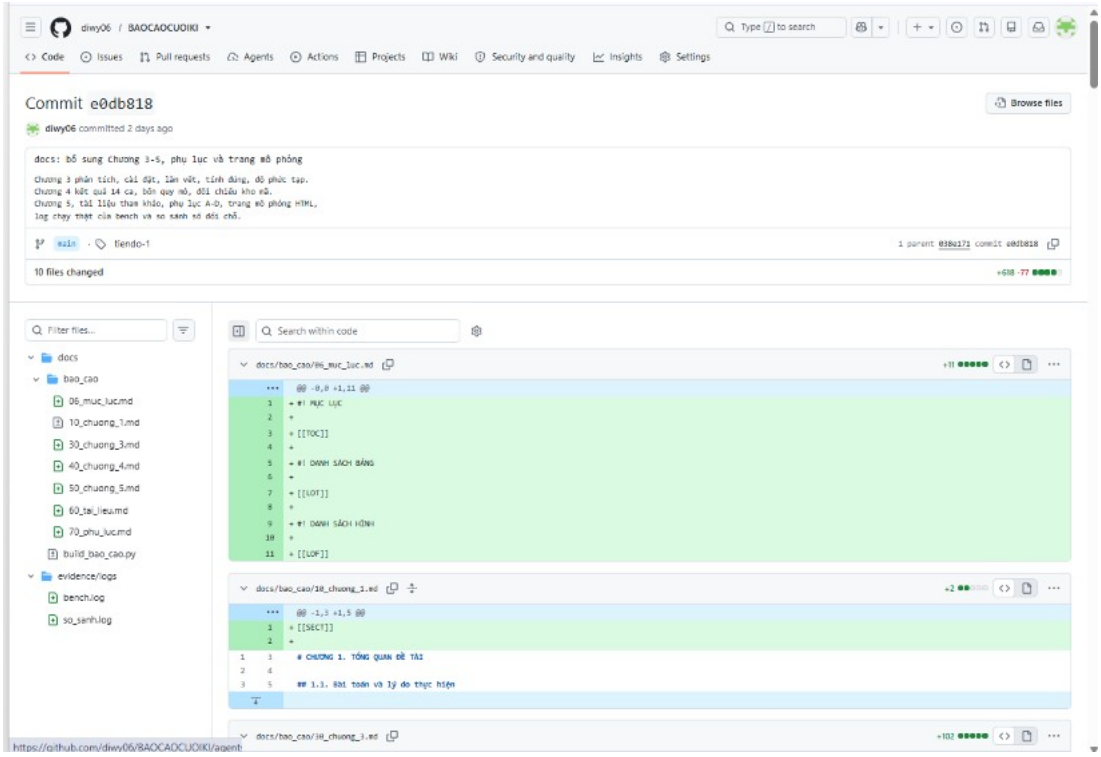
Ảnh A10 cho thấy thư mục clone mới hoàn toàn, HEAD = e0db818 đúng như khai báo. Chạy lại bin/dt011.jl check cho 14/14 ca PASS, 137 phép kiểm đạt.



Hình 4.10 — Clone từ GitHub tại commit C (e0db818): check 14/14 ca, 137 phép kiểm đạt.

4.9.3. Bằng chứng kho GitHub thật

Ảnh A11 là trang commit e0db818 trên GitHub: thanh địa chỉ https://github.com/diwy06/BAOCAOCUOIKI/commit/e0db818…, tiêu đề commit, tác giả và danh sách 10 tệp thay đổi. Đối chiếu với Bảng 4.5 thấy URL, SHA và tệp khớp nhau.



Hình 4.11 — Trang commit C (e0db818) trên GitHub: 10 tệp thay đổi.

4.10. Mô phỏng trực quan (tuỳ chọn)

Ngoài mã lõi, đồ án có thêm trang mô phỏng tương tác docs/MoPhong_SelectionSort_DT011.html — một tệp HTML tự chứa, mở trực tiếp bằng trình duyệt, không cần cài đặt hay mạng. Trang minh hoạ: (i) giả mã có dòng đang chạy được tô sáng; (ii) chạy đối đầu song song sắp xếp lựa chọn và sắp xếp đổi chỗ trên cùng mảng, đếm số đổi chỗ theo thời gian thực; (iii) biểu đồ số so sánh n(n-1)/2 với n = 5, 20, 100, 500; (iv) chế độ sắp giảm dần (chọn max); (v) sao chép bảng truy vết.

Trang mô phỏng chỉ là công cụ quan sát thuật toán, không phải bằng chứng chạy; mọi số liệu trong báo cáo vẫn lấy từ mã Julia tại commit C. Trang dùng cùng quy ước với lõi (đếm n-i phép so sánh mỗi lượt, chỉ đổi khi vị trí min khác i) nên kết quả hiển thị trùng với bin/dt011.jl.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu mục tiêu Chương 1: các hàm sap_xep_lua_chon, tim_min_doan, lan_vet_lua_chon và đối chứng sap_xep_doi_cho đã cài đặt và chạy đúng; 14/14 ca kiểm thử PASS, 137 phép kiểm đạt; 11 ảnh minh chứng và kho mã chạy lại được.

5.2. Giới hạn và bài học

Giới hạn: thuật toán O(n^2) nên chỉ phù hợp n ≤ 2000 như phiếu quy định; không phải thuật toán nhanh cho dữ liệu lớn. Một lỗi tiêu biểu đã gặp là quên điều kiện min_idx != i khiến đổi chỗ một ô với chính nó bị tính nhầm; phát hiện nhờ bảng truy vết và ca BIEN04 ([1,2,3] đã tăng), sau đó sửa và chạy lại.

5.3. Hướng phát triển phù hợp

Hướng phát triển: (1) dùng heap để giảm thời gian tìm min xuống O(log n), đưa về O(n log n) — chưa cài đặt, là hướng mở rộng. Riêng yêu cầu tùy chọn của phiếu (sắp giảm dần bằng cách chọn max thay min) thì đã cài đặt: tham số giam=true của hàm sap_xep_lua_chon, có testset “Yeu cau tuy chon: sap giam dan (chon max)” trong spec/runtests.jl.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Cài đặt sắp xếp lựa chọn	Hoàn thành	Chương 3, Hình 4.1 –4.4	O(n^2)
Đếm so sánh/đổi chỗ, đối chứng	Hoàn thành	Bảng 4.4, mục 4.4.2, evidence/logs/so_sanh.log	—
Bộ test ≥12 ca, 11 ảnh	Hoàn thành	Bảng 4.2, 4.3; A01–A11	A11 là trang commit C trên GitHub

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; Chương 2, Mục 2.2.2; trang in 35–37; trang PDF 47–49.

[2] Mã mẫu BFS của giảng viên (Ma_BFS_mau.zip) — chỉ tham khảo cách tổ chức module, demo, test và chụp ảnh; thuật toán và kiểm thử là tự cài cho đề DT011.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Cấu trúc thư mục nộp (Julia 1.13.1, chỉ thư viện chuẩn):

```
N24DCDT022_DT011_TranHuuDuy/
  bin/dt011.jl                 CLI: demo, trace, bench, check, input
  src/DT011.jl                 module loi: validator, tim min, sap xep, lan vet
  spec/cases.jl                14 ca kiem thu (du lieu)
  spec/runtests.jl             bo kiem thu doc lap
  bench/scales.jl              khao sat n = 5, 20, 100, 500
  evidence/images/             11 anh chup that
  evidence/logs/               log chay that
  Run_all.ps1                  chay toan bo bang mot lenh
  docs/MoPhong_SelectionSort_DT011.html   mo phong tuong tac
# Vi du 1
julia --startup-file=no bin/dt011.jl demo --case VD01
# Vi du 2
julia --startup-file=no bin/dt011.jl demo --case VD02
# Nhap du lieu moi
julia --startup-file=no bin/dt011.jl input
# Bo kiem thu 14 ca
julia --startup-file=no bin/dt011.jl check
# Khao sat bon quy mo
julia --startup-file=no bin/dt011.jl bench --scales 5,20,100,500 --seed 20261020
```

PHỤ LỤC B. Danh mục bằng chứng

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh và log minh chứng
A01	VD01	01_VD01.png
A02	VD02	02_VD02.png
A03	TK01	03_TK01.png
A04	TK02	04_TK02.png
A05	BIEN01	05_BIEN01.png
A06	BIEN02	06_BIEN02.png
A07	BIEN03	07_BIEN03.png
A08	LOI01	08_LOI01.png
A09	CHẠY LẠI TRÊN S	09_check.png
A10	CLONE TỪ GITHUB	10_clone_GitHub.png
A11	TRANG COMMIT GITHUB	11_github.png

Ghi chú: ngoài 11 ảnh trên, gói mã nguồn còn tệp 12_trace_VD01.png là ảnh lần vết VD01 để tham khảo, không tính trong 11 ảnh minh chứng.

PHỤ LỤC C. Tự kiểm trước khi nộp

[x] MSSV và đề tài đúng bảng phân công (N24DCDT022 / DT011).

[x] Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.

[x] Đủ 14 ca (13 ca dữ liệu + LOI01); 11 ảnh thật A01–A11: A05–A07 là ba ca biên, A10 là ảnh clone từ GitHub, A11 là trang commit.

[x] Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và bốn quy mô n = 5, 20, 100, 500.

[x] URL, SHA C, permalink và quyền truy cập kiểm được; clone tại C chạy lại 14/14 ca.

[x] Đã rà lại các mục, cập nhật mục lục và xuất bản PDF từ bản Word.

PHỤ LỤC D. THÔNG TIN NỘP BÀI

Bộ tệp nộp gồm bốn tệp: báo cáo Word, báo cáo PDF, ZIP mã nguồn và TXT thông tin nộp; tên tệp theo tiền tố N24DCDT022_DT011_TranHuuDuy.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT022_DT011_TranHuuDuy_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	N24DCDT022_DT011_TranHuuDuy_BaoCao.pdf	Xuất từ chính bản Word nộp.
Mã nguồn	N24DCDT022_DT011_TranHuuDuy_MaNguon.zip	Source, demo, test, README, ảnh/log.
Thông tin	N24DCDT022_DT011_TranHuuDuy_ThongTinNop.txt	URL GitHub, SHA C/S, permalink, lệnh.

ZIP mã nguồn chứa mã, kiểm thử, README, ảnh và log; giải nén ra thư mục mới chạy được theo README, không kèm .git hay thư mục cài đặt.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh và báo cáo. Clone tại C chạy lại cho 14/14 ca PASS; SHA S ghi trong tệp TXT nộp rời.

Thông tin nộp kèm trong tệp TXT

URL GitHub thật: https://github.com/diwy06/BAOCAOCUOIKI

SHA C đầy đủ: e0db818fda1007c79bda71b8076cf6ccc7fa73a8 SHA S: 2dbb6921d01124c6d9a1897afa97aace1e53654c

Permalink hàm chính: https://github.com/diwy06/BAOCAOCUOIKI/blob/e0db818fda1007c79bda71b8076cf6ccc7fa73a8/src/DT011.jl Permalink test: https://github.com/diwy06/BAOCAOCUOIKI/blob/e0db818fda1007c79bda71b8076cf6ccc7fa73a8/spec/runtests.jl

Môi trường và phiên bản: Julia 1.13.1, Windows

Lệnh chạy hai ví dụ và bộ test: julia --startup-file=no bin/dt011.jl demo --case VD01; julia --startup-file=no bin/dt011.jl demo --case VD02; julia --startup-file=no bin/dt011.jl check

Kết quả clone tại C: chạy lại PASS (14/14 ca)